# Rapport de projet

## Analyse et comparaison des algorithmes de clustering

*K-Means • GMM • DBSCAN — Dataset Iris*

Professeure : Nadjate Saïdani

**Équipe :**
**Cheroufa Aissa**
**Hanafi Ahcene**
**Kaci Sirem**

# 1. Introduction

Ce rapport présente les résultats du projet portant sur la comparaison de trois algorithmes de clustering non supervisés : K-Means, le modèle de mélange gaussien (GMM) et DBSCAN. Ces algorithmes sont appliqués au jeu de données Iris, un classique de l'apprentissage automatique.

Le projet comprend trois volets principaux :

- La présentation théorique de l'algorithme GMM.
- L'implémentation manuelle de DBSCAN en Python, sans utiliser scikit-learn.
- L'expérimentation des trois algorithmes sur le dataset Iris, avec analyse des hyperparamètres.

# 2. Présentation du dataset Iris

Le dataset Iris contient 150 observations décrites par quatre variables numériques : longueur et largeur des sépales, longueur et largeur des pétales. Il ne contient aucune valeur manquante et un seul doublon.

Avant d'appliquer les algorithmes, les données ont été standardisées (moyenne ≈ 0, écart-type ≈ 1) afin d'éviter que certaines variables dominent les calculs de distance.

# 3. Présentation théorique de GMM

Le Gaussian Mixture Model (GMM) est une méthode de clustering probabiliste. Contrairement à K-Means qui attribue chaque point à un seul cluster, GMM estime la probabilité qu'un point appartienne à chacun des clusters.

Chaque cluster est représenté par une loi gaussienne définie par :

- Une moyenne $\mu$ (centre du cluster)
- Une matrice de covariance $\Sigma$ (forme et orientation)
- Un poids $\pi$ (importance relative du cluster)

Le modèle fonctionne de façon itérative grâce à l'algorithme EM (Expectation-Maximization) :

- Étape E : calcul des probabilités d'appartenance de chaque point à chaque composante gaussienne.
- Étape M : mise à jour des paramètres des gaussiennes à partir de ces probabilités.

GMM est plus flexible que K-Means car il peut modéliser des clusters de formes elliptiques ou de tailles différentes.

# 4. Implémentation manuelle de DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) est un algorithme de clustering basé sur la densité. Il utilise deux hyperparamètres :

- eps : rayon du voisinage d'un point.
- min_samples : nombre minimum de voisins pour qu'un point soit considéré comme point cœur.

L'algorithme distingue trois types de points : les points cœurs (assez de voisins), les points frontières (dans le voisinage d'un point cœur) et le bruit (points isolés).

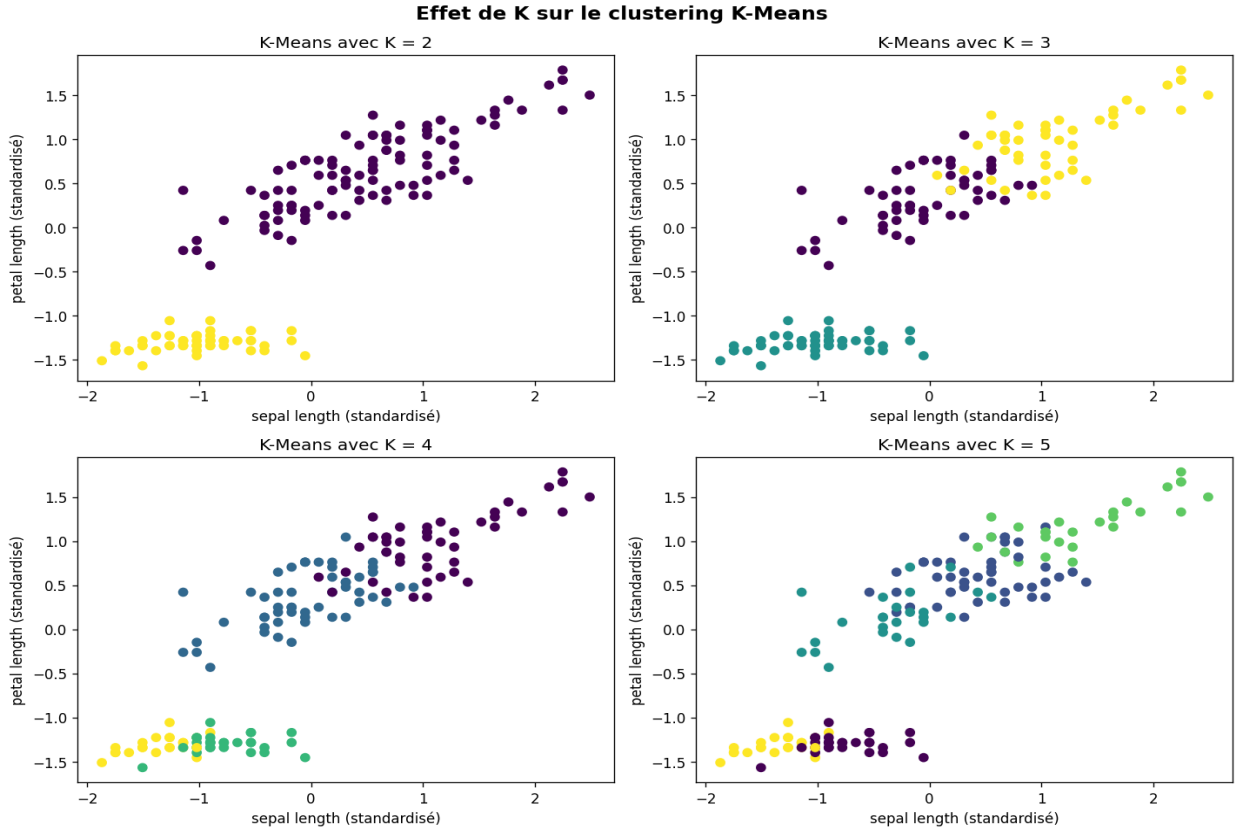
Une implémentation simplifiée a été réalisée en Python pur, sans scikit-learn. Le test rapide avec eps = 0.8 et min_samples = 5 a donné les résultats suivants :

| Cluster | Nombre de points | Interprétation |
|---|---|---|
| -1 (bruit) | 4 | Points isolés |
| Cluster 0 | 49 | Zone dense |
| Cluster 1 | 97 | Zone dense |

# 5. Expérimentation avec K-Means

## 5.1 Effet du paramètre K

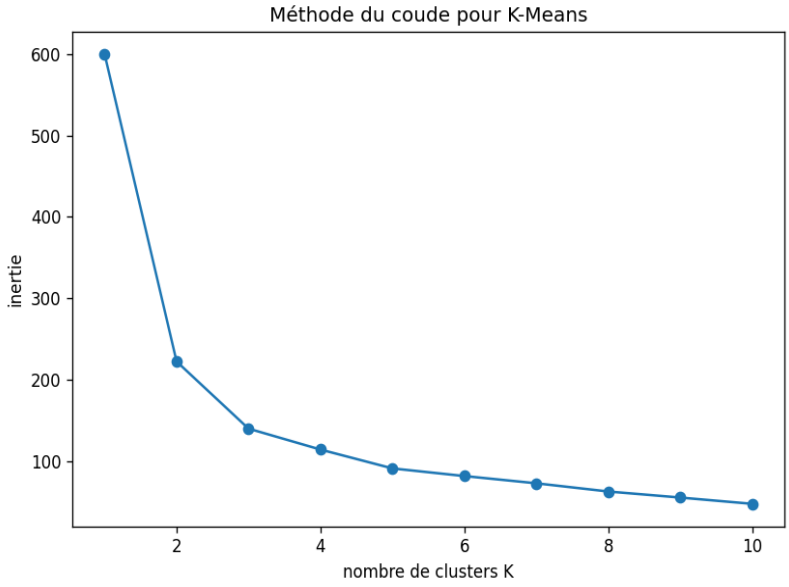
Quatre valeurs de K ont été testées (2, 3, 4, 5). Les graphiques ci-dessous montrent comment la structure des clusters évolue.




Avec K = 2, les groupes sont trop généraux. K = 3 donne une séparation plus équilibrée. Pour K = 4 et K = 5, les clusters deviennent plus nombreux mais moins naturels visuellement.

## 5.2 Méthode du coude

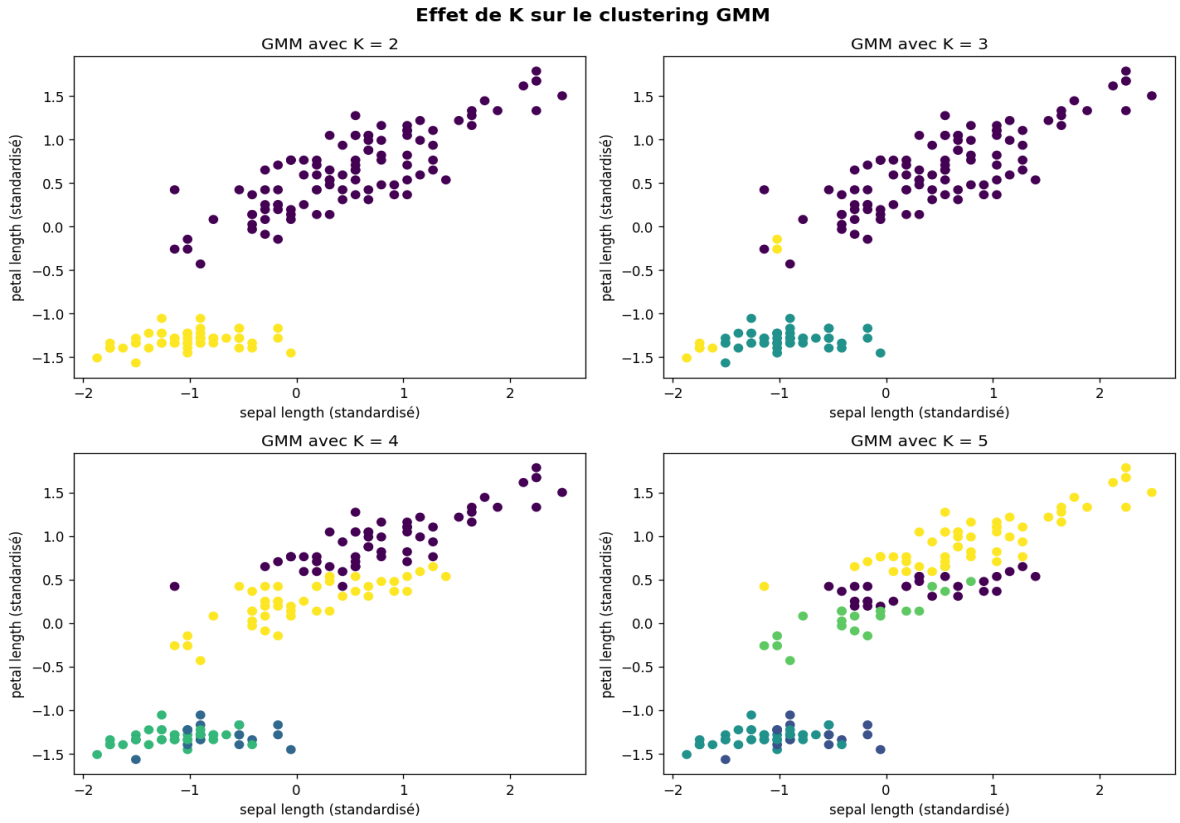
La méthode du coude confirme que la baisse d'inertie ralentit significativement à partir de K = 3, qui est donc la valeur retenue.




# 6. Expérimentation avec GMM

## 6.1 Effet du nombre de composantes K

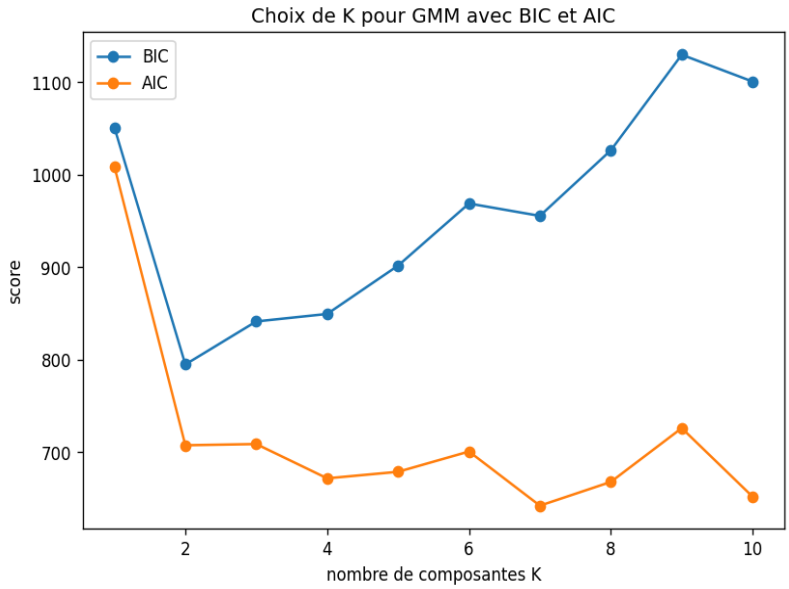
Les mêmes valeurs de K ont été testées avec GMM. Le résultat est visuellement similaire à K-Means, mais les frontières entre clusters sont plus souples.




K = 4 a été retenu pour GMM, car il offre une partition plus fine et plus intéressante que K = 3, sans être trop fragmentée comme K = 5.

## 6.2 Critères BIC et AIC

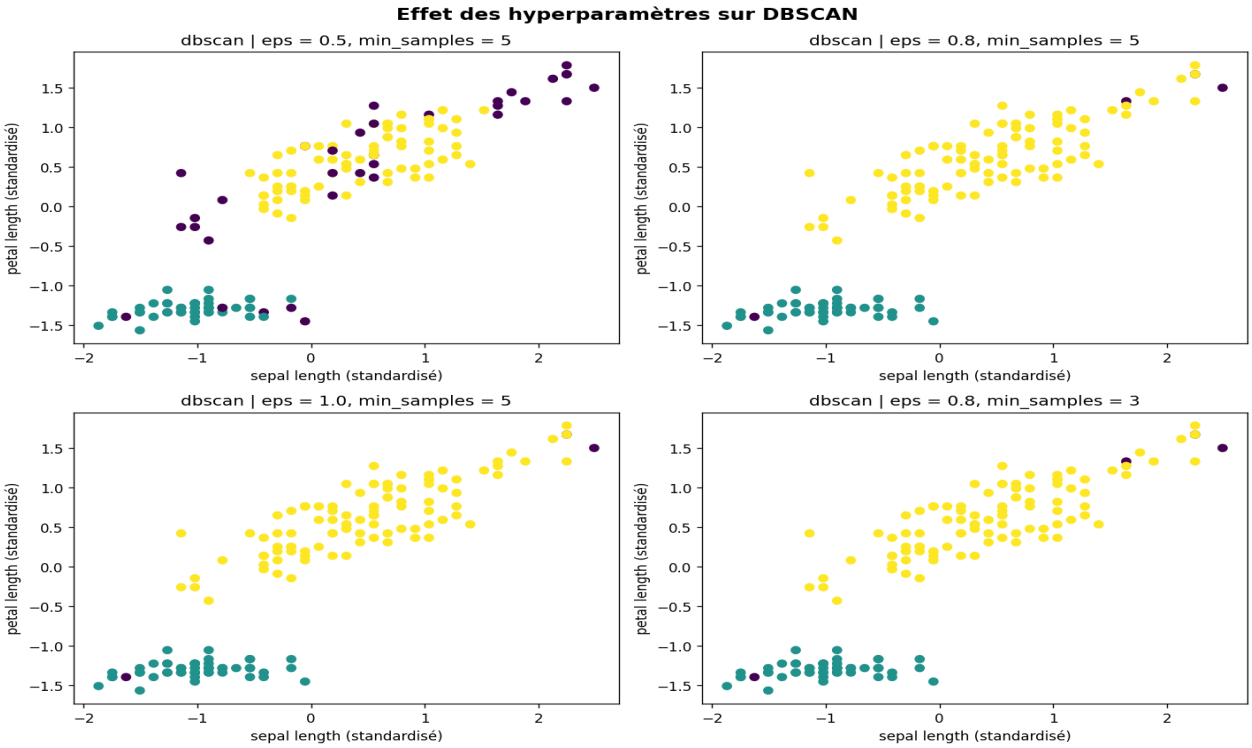
Les critères BIC et AIC permettent de comparer la qualité du modèle selon le nombre de composantes.




Le BIC favorise K = 2 (score minimum), tandis que l'AIC suggère K = 7. En combinant ces critères avec l'analyse visuelle, K = 4 a été retenu comme bon compromis.

# 7. Expérimentation avec DBSCAN

Quatre combinaisons de paramètres ont été testées pour analyser leur effet sur les clusters et le bruit détecté.

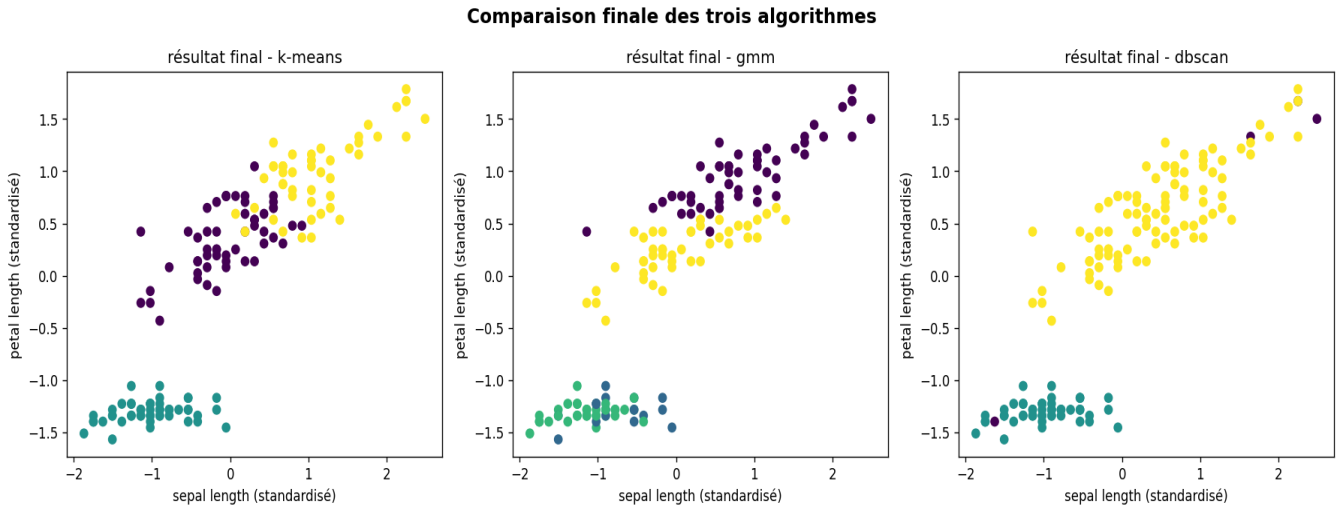


| eps | min_samples | Clusters | Points bruit |
|---|---|---|---|
| 0.5 | 5 | 2 | 34 |
| 0.8 | 5 | 2 | 4 |
| 1.0 | 5 | 2 | 3 |
| 0.8 | 3 | 2 | 4 |

L'hyperparamètre eps a ici plus d'impact que min_samples. Un eps trop petit (0.5) génère beaucoup de bruit. La valeur eps = 0.8 avec min_samples = 5 offre le meilleur équilibre.

## 8. Comparaison finale

La figure ci-dessous présente les résultats retenus pour chaque algorithme sur le même espace de données.




| Algorithme | Hyperparamètres | Nb clusters | Bruit | Remarque |
|---|---|---|---|---|
| K-Means | K = 3 | 3 | Non | Simple |
| GMM | K = 4 | 4 | Non | Flexible |
| DBSCAN | eps=0.8, min_s=5 | 2 | Oui (4 pts) | Densité |

## 9. Conclusion

Ce projet a permis de comparer trois approches complémentaires du clustering non supervisé :

- K-Means est l'algorithme le plus simple et le plus rapide. Avec K = 3, il donne une séparation claire et cohérente, adaptée aux groupes relativement sphériques.
- GMM est plus flexible grâce à son approche probabiliste. Avec K = 4, il produit une partition plus fine, capable de détecter des groupes de formes variées.
- DBSCAN est le seul à détecter du bruit. Il ne nécessite pas de spécifier le nombre de clusters à l'avance, mais il est très sensible au paramètre eps.

Ces résultats illustrent qu'il n'existe pas d'algorithme universel : le choix dépend de la structure des données, des contraintes du problème et de l'interprétation souhaitée.